

Exploring Recurrent Neural Networks (RNNs) : Have Informed Investment w/ Stock Price Prediction, an Advice on AMZN & IBM Stocks

Charles Fredric G. Inventado
College of Information and Computing Sciences
University of Santo Tomas
Bulacan, Philippines
charlesfredric.inventado.cics@ust.edu.ph

James Vincent V. Valles
College of Information and Computing Sciences
University of Santo Tomas
Mandaluyong, Philippines
jamesvincent.valles.cics@ust.edu.ph

Abstract— Understanding the trends, patterns, and correlations of Stock Price Predictions given the current economy of a company's stocks is critical for investors in order to make informed decisions which aims to maximize the profit while minimizing the financial risks. The research explores Amazon and IBM stock price history from years 2006 to 2018, including metrics that help the investors analyze market movements such as open, high, low, close, and volume along with company name and the recorded date. Given the dataset and the sequential data, researchers implemented a Recurrent Neural Network (RNN) model that could perform forecasts of a current year stock price given all previous years stock prices history. Final Model involving GRU-RNN Model with 50 and 20 units for AMZN and IBM respectively, with Adam optimizer, added rolling average before the use of MinMaxScaler(). Results on final model showed a strong potential on predicting stock prices the most, with above 0.98 R^2 value on both Amazon and IBM datasets, 13.80 RMSE and 190.35 MSE on AMZN dataset while performed excellently on IBM dataset with only 1.68 RMSE and 2.81 MSE. Surprisingly, the testing results performed better than the final model when compared to the training results. Researchers suggest for future studies to find solutions and verify whether they encounter common Data Science problems such as data leakage, underfitting, unoptimized regularization, and appropriate normalization and scaling procedures.

Keywords— Recurrent Neural Networks, Stock Prices Prediction, Amazon Stocks, IBM Stocks, Dropout, LSTM, GRU, Bi-RNN, Regression Model, Normalization, Scaling, Hyperparameter Tuning, Sequence Lengths, Data Imputation, Missing Values, Preprocessing, Learning Rate, Adding Layers, Hidden Size, Adam, Feature Engineering, Date, Open, High, Close, Volume, Company, R^2 , MSE, RMSE

I. INTRODUCTION

Currently on Summative Lab Exercise #4 revolves on Recurrent Neural Networks (RNN) which is designed to be used for sequential data such as texts or sentences rather than time-series predictions. RNN is distinguished by having a memory of previous inputs [1] which is then used to model the

probabilities to have an output of a text predicted or a value predicted. Thus, a difference on a current output may occur if there is a change on the previous inputs, hidden states, and the current input step. Whereas the previous lab task such as Convolutional Neural Networks (CNN) will struggle specifically on catching patterns that contain dependencies related to temporal data. [2]

A. Overview of RNNs

Understanding further on how the data makes sense in an sequential order, making an example on text speech, RNN tracks the meaning or context by maintaining a hidden state at each time state. Where a feedback loop is also established on each step then to the next. Whereas a current input and a hidden state from previous time step allows RNN to still remember the previous data and use these to determine the current output. [2]

Example scenario where RNNs are useful on texts “Beat around the bush” for example where each word positioned has significant information as an idiom to determine the next possible word on that sequence. [2]

For example, the first scenario where RNNs are useful in the field of Natural Language Processing where Long Short-Term Memory Networks (LSTMs) and Gated Recurrent Unit (GRUs) are used for text generation, translation, and sentiment analysis. Second, speech recognition, where we speak and the speech will be converted into text whereas virtual assistants such as Siri are able to do tasks based on the converted text. [1] The third field is the current problem on summative lab exercise 4, which will be discussed in the next sub-section.

B. Problem

The third scenario where RNNs are useful and the researchers must resolve the problem from is the time-series forecasting where available sequential data are used to forecast trends. Specifically, the researchers have to make a regression model that could predict the future stock prices of Amazon and International Business Machines Corporation (IBM) from 2006 to 2018 given the previous data.

Thus, researchers will implement a baseline RNN model which will serve as a basis for comparison when experiments are conducted where there will be a change in approach on the RNN model on each different experiments such as on including more preprocessing methods, adding layers, changing RNN type, changing number of RNN number of units, and other potential improvements that could lead to experiment of combined approaches that could result to the best performing RNN model in terms of metrics to be used on the study such as MSE, RMSE, and R^2 .

C. Dataset Overview

Given there are two different companies mentioned on the problem, thus, datasets containing stock price history of IBM and Amazon from 2006 to 2018 separated and stored from two .csv files. Then experiments will be applied to both of the datasets separately as well.

The dataset is provided by the professor, Mr. Jessie James Suarez, as part of summative lab exercise # 4 during the finals period on the course 'Machine Learning'.

Both of the dataset shares the similar column attributes, described as the following labels on *table 1* :

TABLE 1
2006 Amazon & IBM Stock Price Info Column Labels

Column Name	Data Type	Description
Date	-	The recorded date of a company stock price with a format of YYYY-MM-DD
Open	float64	The price of a stock of a company at the beginning of the trading session.
High	float64	The highest price the stock of a company reached during the trading session.
Low	float64	The lowest price the stock of a company reached during the trading session.
Close	float64	The final price of the stock of a company at the end of the trading session
Volume	int64	The total number of shares of a company traded during the session.

Name	object	The company involved in the trading to be invested upon.
------	--------	--

The preprocessing methods of the dataset such as handling missing values and duplicate records will be discussed on methodology.

D. Objectives

In order to identify if our study is a success we've listed down the objectives to aim for when conducting the experiment and its outcomes.

- Building an RNN model from scratch to handle sequential data.
- Experimenting with different hyperparameters, including the number of layers, number of hidden units, dropout rates, learning rates, and sequence lengths.
- Avoiding the use of pre-trained models or architectures

II. METHODOLOGY

The Methodology section discusses how the recurrent neural network will be built from scratch, exploring different components that can be manipulated and implemented on a RNN architecture. The section also includes the different approaches made by the researchers to empirically evaluate the different hyperparameters in preparation of the experimentation of the neural network.

These include the data preprocessing, creation of the CNN model, training of the mode, and detailed performance metrics on the evaluation of the neural network.

A. Data Preprocessing

Handling Missing Values

```
: print("Missing values in Amazon dataset:")
print(amzn_dataset.isnull().sum())

print("\nMissing values in IBM dataset:")
print(ibm_dataset.isnull().sum())

Missing values in Amazon dataset:
Open      0
High      0
Low       0
Close     0
Volume    0
Name      0
dtype: int64

Missing values in IBM dataset:
Open      1
High      0
Low       1
Close     0
Volume    0
Name      0
dtype: int64
```

Fig. 1. Checking Missing Values on Amazon & IBM Datasets

From Fig. 1, Amazon stock price history dataset do not have any missing values, meanwhile one record on IBM stock price history dataset contains null values.

```
missing_ibm = ibm_dataset[ibm_dataset.isnull().any(axis=1)]
print("Rows in IBM with missing values:")
print(missing_ibm)
```

Date	Open	High	Low	Close	Volume	Name
2017-07-31	NaN	144.93	NaN	144.67	4355718	IBM

Fig. 2. The Record with Missing Values on the IBM Dataset

From Fig. 2, the record for 2017-07-31 on IBM dataset does indeed have null values on open and low columns.

Dates in Amazon but NOT in IBM & Dates in IBM but NOT in Amazon

```
amzn_dates = set(amzn_dataset.index)
ibm_dates = set(ibm_dataset.index)

# Dates present in Amazon but not in IBM
only_in_amzn = sorted(amzn_dates - ibm_dates)
# Dates present in IBM but not in Amazon
only_in_ibm = sorted(ibm_dates - amzn_dates)

# Format the dates
only_in_amzn = [date.strftime('%Y-%m-%d') for date in only_in_amzn]
only_in_ibm = [date.strftime('%Y-%m-%d') for date in only_in_ibm]

print("Dates in Amazon but NOT in IBM:")
print(only_in_amzn)

print("\nDates in IBM but NOT in Amazon:")
print(only_in_ibm)
```

Dates in Amazon but NOT in IBM:
[]

Dates in IBM but NOT in Amazon:
['2010-04-01']

Fig. 3. Results of Dates in Amazon but not in IBM and Dates in IBM but not in Amazon

From Fig. 3, Amazon dataset does not have a record for 2010-04-01, but IBM dataset has one record for the specified day.

Handling Duplicate Rows

```
print(f"Amazon Duplicated Rows: {amzn_dataset.duplicated().sum()}")
print(f"IBM Duplicated Rows: {ibm_dataset.duplicated().sum()}")
```

Amazon Duplicated Rows: 0
IBM Duplicated Rows: 0

Fig. 4. Results of Duplicate Row Counts for Amazon & IBM Datasets

From Fig. 4, both Amazon and IBM datasets do not have any duplicate records.

Debate upon Data Preprocessing Actions

From the perspective of the researchers, there is a crucial difference between comparing the two models sensibly if they have different record counts.

Dataset Described

```
print(f"AMZN Dataset Aggregates:\n{amzn_dataset.describe()}")
```

	Open	High	Low	Close	Volume
count	3019.000000	3019.000000	3019.000000	3019.000000	3.019000e+03
mean	299.335310	302.371163	296.037695	299.376231	5.931712e+06
std	280.120547	281.826442	277.927134	279.980161	5.122034e+06
min	26.090000	26.300000	25.760000	26.070000	9.864350e+05
25%	81.175000	82.580000	79.725000	81.090000	3.137037e+06
50%	205.330000	208.000000	202.100000	205.440000	4.724100e+06
75%	375.570000	379.155000	373.000000	375.140000	7.135246e+06
max	1204.880000	1213.410000	1191.150000	1195.830000	1.044046e+08

```
print(f"IBM Dataset Aggregates:\n{ibm_dataset.describe()}")
```

	Open	High	Low	Close	Volume
count	3019.000000	3020.000000	3019.000000	3020.000000	3.020000e+03
mean	145.515545	146.681738	144.471597	145.617278	5.773301e+06
std	37.554946	37.613446	37.477641	37.529387	3.192831e+06
min	72.740000	73.940000	69.500000	71.740000	2.542560e+05
25%	116.405000	117.765000	115.490000	116.525000	3.622681e+06
50%	149.610000	150.330000	148.470000	149.315000	4.928852e+06
75%	178.445000	179.762500	177.330000	178.685000	6.965014e+06
max	215.380000	215.900000	214.300000	215.800000	3.077428e+07

Fig. 5. Description of Amazon and IBM Datasets

From Fig. 5, the record of IBM is actually 3020 with respect to the null values of one record ‘2017-07-31’ as seen from Fig. 2 . Meanwhile from Fig. 3, the Amazon dataset is missing a record of ‘2010-04-01’ .

The researchers took some time to ponder. (and brain cells were confused as well) After some hours, decisions have been made such as filling in information for the NaN values of open and low columns on ‘2017-07-31’ with information available to the researchers. Using advantage on fig. 5 has mean value of different stock prices metrics. Meanwhile on Amazon dataset, to add record ‘2010-04-01’ and have fill-up values columns of means on open, high, low, close, and volume present on Fig. 5 on Amazon dataset.

These decisions are the most suitable decisions from the perspective of the researchers to make where the filled-up values will not cause issues and to ensure the sequential consistency where all dates when compared are similar.

For clarity, after the preprocess, both Amazon and IBM will both have 3020 records after. Which makes more sense to compare 2 different models containing the same dates, without dropping any present records on the dataset.

```

mean_values = amzn_dataset[['Open', 'High', 'Low', 'Close', 'Volume']].mean()

# Create the new row with mean values
missing_date = pd.to_datetime('2010-04-01')
new_row = pd.Series({
    'Open': mean_values['Open'],
    'High': mean_values['High'],
    'Low': mean_values['Low'],
    'Close': mean_values['Close'],
    'Volume': mean_values['Volume'],
    'Name': 'AMZN'
}, name=missing_date)

# Add the row to the dataset
amzn_dataset.loc[missing_date] = new_row

# Sort the index chronologically
amzn_dataset.sort_index(inplace=True)

# Verify
print(amzn_dataset.loc['2010-04-01'])

```

	Open	High	Low	Close	Volume	Name
	299.33531	302.371163	296.037695	299.376231	5931712.40212	AMZN

Name: 2010-04-01 00:00:00, dtype: object

Fig. 6. Values to be Added on Record '2010-04-01' on AMZN dataset

Process to add a record to match 3020 records of IBM from Fig. 6. Lastly, also the NaN values on open and low columns on record '2017-07-31' undergo through the similar process.

```

print(f"AMZN Dataset Aggregates:\n{amzn_dataset.describe()}")
print(f"IBM Dataset Aggregates:\n{ibm_dataset.describe()}")

```

AMZN Dataset Aggregates:						
	Open	High	Low	Close	Volume	
count	3020.000000	3020.000000	3020.000000	3020.000000	3.020000e+03	
mean	299.33531	302.371163	296.037695	299.376231	5.931712e+06	
std	280.07415	281.779763	277.881100	279.933788	5.121185e+06	
min	26.09000	26.300000	25.760000	26.070000	9.864350e+05	
25%	81.18250	82.605000	79.727500	81.095000	3.137116e+06	
50%	205.44000	208.300000	202.175000	205.755000	4.725560e+06	
75%	375.49500	379.077500	372.925000	375.140000	7.134578e+06	
max	1204.88000	1213.410000	1191.150000	1195.830000	1.044046e+08	

IBM Dataset Aggregates:						
	Open	High	Low	Close	Volume	
count	3020.000000	3020.000000	3020.000000	3020.000000	3.020000e+03	
mean	145.515545	146.681738	144.471597	145.617278	5.773301e+06	
std	37.548726	37.613446	37.471433	37.529387	3.192831e+06	
min	72.740000	73.940000	69.500000	71.740000	2.542560e+05	
25%	116.407500	117.765000	115.500000	116.525000	3.622681e+06	
50%	149.605000	150.330000	148.425000	149.315000	4.928852e+06	
75%	178.437500	179.762500	177.320000	178.685000	6.965014e+06	
max	215.380000	215.900000	214.300000	215.800000	3.077428e+07	

Fig. 7. Same Amount of Records and Dates Included on Amazon and IBM Datasets

Afterall, the preprocessing for the missing values and records are complete from Fig. 7.

Data Splitting



Fig. 8. Data Splitting on Amazon Stock Price Dataset



Fig. 9. Data Splitting on IBM Stock Price Dataset

From Fig. 8 and Fig. 9, data splitting have been conducted with the training dataset includes records from 2006 to before 2017 while the test set has the records starting year 2017 and beyond to 2018.

Scaling

```

# Scaling the training set
sc_rms_amzn = MinMaxScaler(feature_range=(0,1))
amzn_training_set_scaled = sc_rms_amzn.fit_transform(amzn_training_set)

sc_rms_ibm = MinMaxScaler(feature_range=(0,1))
ibm_training_set_scaled = sc_rms_ibm.fit_transform(ibm_training_set)

```

Fig. 10. Scaling of Amazon and IBM Datasets

From Fig. 10, the records were then appropriately scaled using MinMaxScaler that will help the RNN model process.

Converting Inputs for RNN

```

def create_lstm_dataset(training_set_scaled, timesteps=60):
    X_train = []
    y_train = []
    for i in range(timesteps, len(training_set_scaled)):
        X_train.append(training_set_scaled[i - timesteps:i, 0])
        y_train.append(training_set_scaled[i, 0])
    X_train, y_train = np.array(X_train), np.array(y_train)
    X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
    return X_train, y_train

amzn_X_train, amzn_y_train = create_lstm_dataset(amzn_training_set_scaled)
print(amzn_X_train.shape)

(2709, 60, 1)

ibm_X_train, ibm_y_train = create_lstm_dataset(ibm_training_set_scaled)
print(ibm_X_train.shape)

(2709, 60, 1)

```

Fig. 11. Scaling of Amazon and IBM Datasets

From Fig. 11, a code on preparation for time-series data on training an Long Short-Term Memory Model (LSTM).

Wherein inputs, training data, and the loop were implemented on the RNN model. Aswell reshaping columns into a format required for LSTM and efficient computation. a code on preparation for time-series data on training an Long Short-Term Memory Model (LSTM). Wherein inputs, training data, and the loop were implemented on the RNN model. Aswell reshaping columns into a format required for LSTM and efficient computation.

B. Model Implementation

```
# The LSTM architecture
amzn_regressor = Sequential()
# First LSTM layer with Dropout regularisation
amzn_regressor.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1],1)))
# Second LSTM layer
amzn_regressor.add(LSTM(units=5))
# The output layer
amzn_regressor.add(Dense(units=1))

# Compiling the RNN
amzn_regressor.compile(
    optimizer='rmsprop',
    loss='mean_squared_error',
    metrics=['mean_squared_error', 'mean_absolute_error']
)

# Fitting to the training set
amzn_regressor.fit(amzn_X_train, amzn_y_train, epochs=10, batch_size=32)
```

Fig. 12. LSTM-Based Recurrent Neural Network Architecture

From Fig. 12, describing the baseline RNN model, uses sequential to have a layered neural network and implements LSTM to retain memory of previous data in a sequential data. The first LSTM LSTM layer also has dropout regularization, having 5 units capturing the dependencies on stock price sequence, return_sequences is True that allows the LSTM layer to receive the sequence outputs instead of only the final state. Second LSTM layer, also 5 units without the return_sequence, as a final recurrent layer that outputs a single hidden state into the next layer. Third is the dense layer with 1 unit, producing the value of a single predicted stock price.

To support the RNN architecture evaluations, rmsprop optimizer is also used to help learning rate adjust dynamically for time-series stability. Loss function is the mean squared error (MSE) where precision matters and error grows larger exponentially as the value goes further from zero. Mean squared error and mean absolute error are also considered to track performance during training

Epochs is equal to 10, meaning the model trains through the dataset 10 times. Then the batch size is 32 wherein 32 samples per training step were processed in chunks.

C. Performance Evaluation

The metrics to be used are according to the type of problem the researcher is solving – which is a prediction of continuous numbers which in this case are stock prices.

Take note that the metrics to be used are similar to the ones used on doing regression analysis because the study analyzes the difference between the actual value compared to the predicted stock price value of the RNN model. Furthermore, determining the results of the predicted continuous values, assess the variance of stock prices, and monitor the trend consistency of the model.

As the objectives of the research also mentioned is a regression problem, these are the following metrics that will aid on evaluating the performance of built RNN model :

1. **MSE** - The average squared difference of the predicted values to the actual values.

The MSE ranges from 0 up to any positive value where 0 indicates that the model has no error. [4]

2. **RMSE** - Average difference between a model's predicted values and actual values. It is known also as the standard deviation of the residuals, which refers to the difference between the regression line and data points. RMSE can range from zero to positive infinity [5]

3. **R²** - Proportion of variance for a dependent variable that is explained by an independent variable. [6]

The value of R² usually ranges from 0 to 1 where 1 indicates a perfect fit to the actual data.

D. Hyperparameters and Other Approach Changes

In preparation for the experiments, these are the following hyperparameters and other approaches that the researchers decided to pursue which are deemed to have potential on improving the stock price prediction.

Parameter / To be Modified	Description

Normalization preprocessing	Instead of using the standard MinMaxScaler() , researchers attempt to use other approaches possible for normalization or scaling of dataset values that might be more appropriate to the RNN.
Number of RNN layers	The depth and ability of the RNN to learn more complex patterns. Adding a number of layers may have potential therefore researchers attempt as an experiment.
Type of RNN	There are different types of RNN such as LSTM, GRU, and Bi-RNN, where each has unique characteristics that could be used to their own advantage and solve different problems that an RNN may have limitations on.
Number of Units or Hidden Size	Refers to how many memory cells each recurrent layer has and can affect the model's learning on the sequential dependencies. More units may allow deeper feature extraction while fewer units may be more suitable for more simple task extraction.
Learning Rate	The control of step size or how much the model weights update during the training. Determines how fast or slow a model can learn.
Time Steps	Refers to the number of previous observations that an RNN can retain or remember for consideration.
Adding Dropouts	Includes a rate or probability that a neuron can be randomly deactivated during training and usually to combat against the model overfitting.
Including a Momentum - Based Optimization	The hyperparameter could accelerate training by computing past gradients to the current update, such that adding a fraction from the previous gradient to the current update. Example is the use of Adam that combines momentum with adaptive learning rates.
Data Augmentation	Refers to the artificial expanding of the dataset with techniques such as jittering and smoothing whereas variability is introduced while still preserving the temporal dependencies.

Feature Engineering	Refers to transforming the original data into meaningful representations that could improve the performance of RNN models. Feature engineering techniques such as rolling statistics, computing the moving average may help on a model's performance.
---------------------	---

III. EXPERIMENTS

The third section details the experimental setup, as well the different hyperparameter tuning processes.

A. Experimental Setup

Baseline Model (B)

```
# The LSTM architecture
ibm_regressor = Sequential()
# First LSTM layer with dropout regularisation
ibm_regressor.add(LSTM(units=5, return_sequences=True, input_shape=(ibm_X_train.shape[1],1)))
# Second LSTM layer
ibm_regressor.add(LSTM(units=5))
# The output layer
ibm_regressor.add(Dense(units=1))

# Compiling the ANN
ibm_regressor.compile(
    optimizer='rmsprop',
    loss='mean_squared_error',
    metrics=['mean_squared_error', 'mean_absolute_error']
)

# Fitting to the training set
ibm_regressor.fit(ibm_X_train,ibm_y_train,epochs=10,batch_size=32)
```

Fig. 13. Code Snippet of Baseline Model

From Fig. 12 and Fig. 13, was the implemented baseline experiment whereas the other experimental set-ups will be compared to the baseline model in order to describe whether the performance on training and testing achieved improvements that could help building the final model with the highest performance later.

Experiment 1 : Different Scaling Technique

```
amzn_sc_ss = StandardScaler()
amzn_training_set_scaled_ss = amzn_sc_ss.fit_transform(amzn_training_set)

ibm_sc_ss = StandardScaler()
ibm_training_set_scaled_ss = ibm_sc_ss.fit_transform(ibm_training_set)
```

Fig. 14. Code Snippet of Experiment 1 (1)

```
amzn_regressor_ex1 = Sequential()
amzn_regressor_ex1.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train_ex1.shape[1],1)))
amzn_regressor_ex1.add(LSTM(units=5))
amzn_regressor_ex1.add(Dense(units=1))

amzn_regressor_ex1 = build_and_train_model(amzn_regressor_ex1, amzn_X_train_ex1, amzn_y_train_ex1)
```

Fig. 15. Code Snippet of Experiment 1 (2)

The first experiment with changes can be seen from Fig. 14 and Fig. 15 involves changing the scaling of the dataset values. From StandardScaler to MinMaxScaler

Where standardscaler data was centered around zero where mean is zero and standard deviation is one. Meanwhile MinMaxScaler has the data scaled to a fixed range between 0 and 1.

Experiment 2 : Adding Layers

```

amzn_regressor_ex2 = Sequential()
amzn_regressor_ex2.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1],1)))
amzn_regressor_ex2.add(LSTM(units=5, return_sequences=True))
amzn_regressor_ex2.add(LSTM(units=5)) # Third 3 Layer LSTM Layer
amzn_regressor_ex2.add(Dense(units=1))

amzn_regressor_ex2 = build_and_train_model(amzn_regressor_ex2, amzn_X_train, amzn_y_train)

```

Fig. 16. Code Snippet of Experiment 2 Regarding Adding RNN Layers

Second experiment as seen on Fig. 16 adds an additional LSTM layer after the second LSTM layer, resulting in a total of three LSTM layers.

The researchers investigate whether deeper networks on an RNN can capture the more complex patterns on the Amazon and IBM datasets.

Experiment 3 : Changing RNN Type

Third experiment explores using different RNN types such as GRU, and Bidirectional. Thus, experiment is three is separate into three subsections :

TABLE 3
Sub-sections of Experiment Three

Section	Description
3.1	Using GRU RNN
3.2	Using Bidirectional RNN

```

# The GRU architecture
amzn_regressor_ex3_1 = Sequential()
amzn_regressor_ex3_1.add(GRU(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1],1)))
amzn_regressor_ex3_1.add(GRU(units=5))
amzn_regressor_ex3_1.add(Dense(units=1))

amzn_regressor_ex3_1 = build_and_train_model(amzn_regressor_ex3_1, amzn_X_train, amzn_y_train)

```

Fig. 17. Code Snippet of Experiment 3.1 using GRU Architecture

```

# The Bidirectional LSTM architecture
ibm_regressor_ex3_2 = Sequential()
ibm_regressor_ex3_2.add(Bidirectional(LSTM(units=5, return_sequences=True, input_shape=(ibm_X_train.shape[1],1))))
ibm_regressor_ex3_2.add(Bidirectional(LSTM(units=5)))
ibm_regressor_ex3_2.add(Dense(units=1))

ibm_regressor_ex3_2 = build_and_train_model(ibm_regressor_ex3_2, ibm_X_train, ibm_y_train)

```

Fig. 18. Code Snippet of Experiment 3.2 using Bi-RNN Architecture

From Table 3, Fig. 17, and Fig. 18, researchers could compare and contrast the three different RNN models and observe which model RNN performed against other RNN model types experimented.

Experiment 4 : Changing Number of Units

Fourth experiment involves the use of a different number of units on the baseline model. There are three different subsections as well which are :

TABLE 4
Sub-sections of Experiment Four

Section	Description
4.1	Using 10 units on an LSTM model
4.2	Using 20 units on an LSTM model
4.3	Using 50 units on an LSTM model

```

amzn_regressor_ex4_1 = Sequential()
amzn_regressor_ex4_1.add(LSTM(units=10, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1)))
amzn_regressor_ex4_1.add(LSTM(units=10))
amzn_regressor_ex4_1.add(Dense(units=1))

amzn_regressor_ex4_1 = build_and_train_model(amzn_regressor_ex4_1, amzn_X_train, amzn_y_train)

```

Fig. 19. Code Snippet of Experiment 4.2 with 10 units or neurons

```

amzn_regressor_ex4_2 = Sequential()
amzn_regressor_ex4_2.add(LSTM(units=20, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1)))
amzn_regressor_ex4_2.add(LSTM(units=20))
amzn_regressor_ex4_2.add(Dense(units=1))

amzn_regressor_ex4_2 = build_and_train_model(amzn_regressor_ex4_2, amzn_X_train, amzn_y_train)

```

Fig. 20. Code Snippet of Experiment 4.2 with 20 units or neurons

```

amzn_regressor_ex4_3 = Sequential()
amzn_regressor_ex4_3.add(LSTM(units=50, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1)))
amzn_regressor_ex4_3.add(LSTM(units=50))
amzn_regressor_ex4_3.add(Dense(units=1))

amzn_regressor_ex4_3 = build_and_train_model(amzn_regressor_ex4_3, amzn_X_train, amzn_y_train)

```

Fig. 21. Code Snippet of Experiment 4.3 with 50 units or neurons

From Table 4, Fig. 19, Fig. 20, and Fig. 21, researchers could compare and contrast the three different numbers of units of 10, 20, and 50, respectively. The researchers want to explore the trade-off between model complexity and overfitting.

Experiment 5 : Changing Learning Rate

Fifth experiment as seen on Fig. 22, involves the use of different learning rates on the baseline model.. Modifying the learning rate to a larger or smaller rate may improve the learning of the RNN model.

```

amzn_optimizer = RMSprop(learning_rate=0.0001)
ibm_optimizer = RMSprop(learning_rate=0.0001)

amzn_regressor_ex5 = Sequential()
amzn_regressor_ex5.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1)))
amzn_regressor_ex5.add(LSTM(units=5))
amzn_regressor_ex5.add(Dense(units=1))

amzn_regressor_ex5 = build_and_train_model(amzn_regressor_ex5, amzn_X_train, amzn_y_train, amzn_optimizer)

```

Fig. 22. Code Snippet of Experiment 5 Regarding Changing Learning Rate

Experiment 6 : Different Sequence Lengths (Time Steps)

Sixth experiment involves the use of different learning rates on the baseline model.. Modifying the learning rate to a larger or smaller rate may improve the learning of the RNN model. Here are the following modifications on time steps :

TABLE 5
Sub-sections of Experiment Six

Section	Description
6.1	Using 30 Time Steps Length
6.2	Using 90 Time Steps Length

```
amzn_X_train_ex6_1, amzn_y_train_ex6_1 = create_training_dataset(amzn_training_set_scaled, 30)
print(amzn_X_train_ex6_1.shape)

(2739, 30, 1)

ibm_X_train_ex6_1, ibm_y_train_ex6_1 = create_training_dataset(ibm_training_set_scaled, 30)
print(ibm_X_train_ex6_1.shape)

(2739, 30, 1)
```

Fig. 23. Code Snippet of Experiment 6.1 with 30 Steps Length

```
amzn_X_train_ex6_2, amzn_y_train_ex6_2 = create_training_dataset(amzn_training_set_scaled, 90)
print(amzn_X_train_ex6_2.shape)

(2679, 90, 1)

ibm_X_train_ex6_2, ibm_y_train_ex6_2 = create_training_dataset(ibm_training_set_scaled, 90)
print(ibm_X_train_ex6_2.shape)

(2679, 90, 1)
```

Fig. 24. Code Snippet of Experiment 6.1 with 90 Steps Length

The current experiment to determine shorter sequences (experiment 6.1 on Fig. 23), medium length (baseline model), or the longer sequences (experiment 6.3 on Fig. 24) will affect the performance on different metrics significantly for the RNN model.

Experiment 7 : Dropout Regularization

On experiment seven and figure below is the Fig. 25, the dropout model is prevented from memorizing training data by deactivating random neurons with a probability using 0.2 or 20% where weights of the dropped neurons are ignored.

```
amzn_regressor_ex7 = Sequential()
amzn_regressor_ex7.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1), dropout=0.2))
amzn_regressor_ex7.add(LSTM(units=5, dropout=0.2))
amzn_regressor_ex7.add(Dense(units=1))
amzn_regressor_ex7 = build_and_train_model(amzn_regressor_ex7, amzn_X_train, amzn_y_train)
```

Fig. 25. Code Snippet of Experiment 7 Regarding Dropout Regularization

Researchers want to see how doing these will aid in preventing overfitting and improve further the generalization or performance of the model on unseen data.

Experiment 8 : Momentum-Based Optimization

```
amzn_optimizer=Adam(learning_rate=0.001)
ibm_optimizer=Adam(learning_rate=0.001)
```

Fig. 26. Code Snippet of Experiment 8 Regarding Momentum-Based Optimization (1)

```
amzn_regressor_ex8 = Sequential()
amzn_regressor_ex8.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train.shape[1], 1)))
amzn_regressor_ex8.add(LSTM(units=5))
amzn_regressor_ex8.add(Dense(units=1))
amzn_regressor_ex8 = build_and_train_model(amzn_regressor_ex8, amzn_X_train, amzn_y_train, amzn_optimizer)
```

Fig. 27. Code Snippet of Experiment 8 Regarding Momentum-Based Optimization (2)

On experiment eight, instead of updating the weights on the current gradient, the momentum includes the past gradients to further smooth the update of weights.

Code snippet from also on Fig. 26 and Fig. 27, the researchers want to check if adaptive learning rate methods such as use of adam may improve training speed and accuracy.

Experiment 9 : Data Augmentation

```
# Add rolling average before scaling
amzn_dataset['Smoothed'] = amzn_dataset['Close'].rolling(window=3).mean()
smoothed_amz_data = amzn_dataset['Smoothed'].dropna().values.reshape(-1, 1)

ibm_dataset['Smoothed'] = ibm_dataset['Close'].rolling(window=3).mean()
smoothed_ibm_data = ibm_dataset['Smoothed'].dropna().values.reshape(-1, 1)

amzn_sc_mms_ex9 = MinMaxScaler()
amzn_training_set_scaled_ex9 = amzn_sc_mms_ex9.fit_transform(smoothed_amz_data)

ibm_sc_mms_ex9 = MinMaxScaler()
ibm_training_set_scaled_ex9 = ibm_sc_mms_ex9.fit_transform(smoothed_ibm_data)
```

Fig. 28. Code Snippet of Experiment 9 Regarding Data Augmentation (1)

```
amzn_X_train_ex9, amzn_y_train_ex9 = create_training_dataset(amzn_training_set_scaled_ex9)
print(amzn_X_train_ex9.shape)

(2958, 60, 1)

ibm_X_train_ex9, ibm_y_train_ex9 = create_training_dataset(ibm_training_set_scaled_ex9)
print(ibm_X_train_ex9.shape)

(2958, 60, 1)

amzn_regressor_ex9 = Sequential()
amzn_regressor_ex9.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train_ex9.shape[1], 1)))
amzn_regressor_ex9.add(LSTM(units=5))
amzn_regressor_ex9.add(Dense(units=1))
amzn_regressor_ex9 = build_and_train_model(amzn_regressor_ex9, amzn_X_train_ex9, amzn_y_train_ex9)
```

Fig. 29. Code Snippet of Experiment 9 Regarding Data Augmentation (2)

Data augmentation techniques such as smoothing apply moving averages or gaussian smoothing that may help prevent overfitting by reducing noise and sharp fluctuations on the time-series data. Meanwhile jittering is adding small random variations to numerical values on the data.

From Fig. 28 and Fig. 29, the aim of the ninth experiment is to simulate real-world data imperfections and improve the robustness of the model. The process done was rolling averages before scaling specifically.

Experiment 10 : Feature Engineering

```
amzn_training_set_ex10 = amzn_dataset[['Open', 'High', 'Low', 'Volume']].values
amzn_test_set_ex10 = amzn_dataset['2017:'][['Open', 'High', 'Low', 'Volume']].values

ibm_training_set_ex10 = ibm_dataset[['Open', 'High', 'Low', 'Volume']].values
ibm_test_set_ex10 = ibm_dataset['2017:'][['Open', 'High', 'Low', 'Volume']].values

def create_training_dataset_ex10(training_set_scaled, timesteps=60):
    X_train = []
    y_train = []
    for i in range(timesteps, len(training_set_scaled)):
        X_train.append(training_set_scaled[i - timesteps:i]) # This is now a vector of 4 features
        y_train.append(training_set_scaled[i]) # This is now a vector of 4 features
    X_train, y_train = np.array(X_train), np.array(y_train)
    X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 4))
    return X_train, y_train
```

Fig. 30. Code Snippet of Experiment 10 Regarding Feature Engineering (1)

```

amzn_features = amzn_dataset[['2016']][['Open', 'High', 'Low', 'Volume']].values
ibm_features = ibm_dataset[['2016']][['Open', 'High', 'Low', 'Volume']].values

amzn_sc_mms_ex10 = MinMaxScaler()
amzn_training_set_scaled_ex10 = amzn_sc_mms_ex10.fit_transform(amzn_features)

ibm_sc_mms_ex10 = MinMaxScaler()
ibm_training_set_scaled_ex10 = ibm_sc_mms_ex10.fit_transform(ibm_features)

amzn_X_train_ex10, amzn_y_train_ex10 = create_training_dataset_ex10(amzn_training_set_scaled_ex10)
print(amzn_X_train_ex10.shape)
(2709, 60, 4)

ibm_X_train_ex10, ibm_y_train_ex10 = create_training_dataset_ex10(ibm_training_set_scaled_ex10)
print(ibm_X_train_ex10.shape)
(2709, 60, 4)

amzn_regressor_ex10 = Sequential()
amzn_regressor_ex10.add(LSTM(units=5, return_sequences=True, input_shape=(amzn_X_train_ex10.shape[1],4)))
amzn_regressor_ex10.add(LSTM(units=5))
amzn_regressor_ex10.add(Dense(units=1))

amzn_regressor_ex10 = build_and_train_model(amzn_regressor_ex10, amzn_X_train_ex10, amzn_y_train_ex10)

```

Fig. 31. Code Snippet of Experiment 10 Regarding Feature Engineering (2)

Tenth experiment from Fig. 30 and Fig. 31, which features engineering methods such as providing more context for prediction, researchers ponder that the process might also help in improving accuracy, therefore added to the list of experiments.

Final Model (F)

```

# Add rolling average before scaling
amzn_dataset['Smoothed'] = amzn_dataset['Open'].rolling(window=3).mean()
smoothed_amz_data = amzn_dataset['Smoothed'].dropna().values.reshape(-1, 1)

ibm_dataset['Smoothed'] = ibm_dataset['Low'].rolling(window=3).mean()
smoothed_ibm_data = ibm_dataset['Smoothed'].dropna().values.reshape(-1, 1)

amzn_sc_mms_final = MinMaxScaler()
amzn_training_set_scaled_final = amzn_sc_mms_final.fit_transform(smoothed_amz_data)

ibm_sc_mms_final = MinMaxScaler()
ibm_training_set_scaled_final = ibm_sc_mms_final.fit_transform(smoothed_ibm_data)

amzn_X_train_final, amzn_y_train_final = create_training_dataset(amzn_training_set_scaled_final, 100)
print(amzn_X_train_final.shape)
(2918, 100, 1)

ibm_X_train_final, ibm_y_train_final = create_training_dataset(ibm_training_set_scaled_final, 30)
print(ibm_X_train_final.shape)
(2988, 30, 1)

# Compile the model with Adam optimizer
amzn_optimizer_final = Adam(learning_rate=0.01)
ibm_optimizer_final = Adam(learning_rate=0.01)

```

Fig. 32. Code Snippet of Final Experiment (1)

```

# Define the model
amzn_regressor_final = Sequential()

# Add LSTM layers
amzn_regressor_final.add(GRU(units=50, return_sequences=True, input_shape=(amzn_X_train_final.shape[1],1)))
amzn_regressor_final.add(GRU(units=50))

# Add Dense Layer for output
amzn_regressor_final.add(Dense(units=1))

# Train the model using preprocessed data (smoothed and scaled)
amzn_regressor_final = build_and_train_model(amzn_regressor_final, amzn_X_train_final, amzn_y_train_final, amzn_optimizer_final)

```

Fig. 33. Code Snippet of Final Experiment (2)

From Fig. 32 and Fig. 33, final model is the culmination of all experiments with a combination of different experiments that excelled, stand out when compared to the baseline model with a goal to perform with the highest metric results at the end of the experiment on predicting stock prices.

IV. RESULTS AND ANALYSIS

The results and analysis section details the evaluation results of every training and testing of different experimented models.

A. Individual Model Performance

Baseline Model (B)

Using Long Short Term Memory or LSTM RNN Architecture

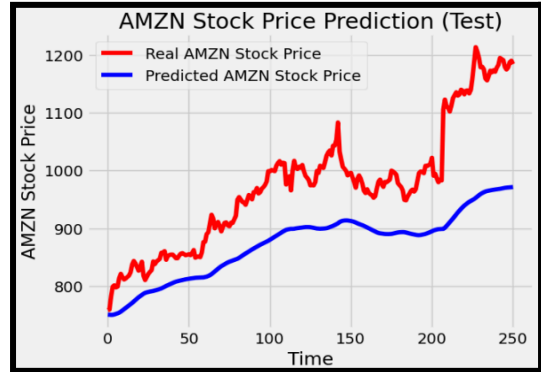


Fig. 34. Baseline Model - AMZN - Testing Predicted vs Actual Stock Price Graph

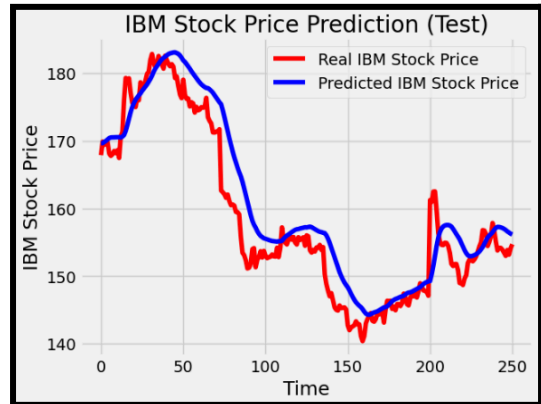


Fig. 35. Baseline Model - IBM - Testing Predicted vs Actual Stock Price Graph

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 116.31333628266715
R-squared (R²): -0.16219524685179243
Mean Squared Error (MSE): 13528.792197204815
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 4.232427170094897
R-squared (R²): 0.8788783298742111
Mean Squared Error (MSE): 17.913439750157497

```

Fig. 36. Baseline Model - AMZN & IBM - Testing Results

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 106.52159283110784
R-squared (R²): 0.7222220281263361
Mean Squared Error (MSE): 11346.849739276324
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.451429765583992
R-squared (R²): 0.6054290747416757
Mean Squared Error (MSE): 597.8724175812869

```

Fig. 37. Baseline Model - AMZN & IBM - Training Results

Experiment 1 : Different Scaling Technique

Experiment 1 using StandardScaler instead of MinMaxScaler

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 174.5679
R-squared (R²): -1.6179
Mean Squared Error (MSE): 30473.9557
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 3.1098
R-squared (R²): 0.9346
Mean Squared Error (MSE): 9.6706

```

Fig. 38. Experiment 1 - AMZN & IBM - Testing Results

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 115.4403
R-squared (R²): 0.6738
Mean Squared Error (MSE): 13326.4616
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.2077
R-squared (R²): 0.6133
Mean Squared Error (MSE): 586.0115

```

Fig. 39. Experiment 1 - AMZN & IBM - Training Results

Experiment 2 : Adding Layers

Experiment 2 adds 1 more LSTM layer, 3 LSTM layers in total.

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 130.5513
R-squared (R²): -0.4641
Mean Squared Error (MSE): 17043.6433
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 5.2984
R-squared (R²): 0.8102
Mean Squared Error (MSE): 28.0730

```

Fig. 40. Experiment 2 - AMZN & IBM - Testing Results

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 113.3834
R-squared (R²): 0.6853
Mean Squared Error (MSE): 12855.8025
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 23.4245
R-squared (R²): 0.6379
Mean Squared Error (MSE): 548.7068

```

Fig. 41. Experiment 2 - AMZN & IBM - Training Results

Experiment 3 : Changing RNN Type

Experiment 3.1 using GRU or Gated Recurrent Unit

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 88.3883
R-squared (R²): 0.3289
Mean Squared Error (MSE): 7812.4905
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 2.7094
R-squared (R²): 0.9504
Mean Squared Error (MSE): 7.3411

```

Fig. 42. Experiment 3.1 - AMZN & IBM - Testing Results

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 111.6094
R-squared (R²): 0.6951
Mean Squared Error (MSE): 12456.6508
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.5395
R-squared (R²): 0.6026
Mean Squared Error (MSE): 602.1892

```

Fig. 43. Experiment 3.1 - AMZN & IBM - Training Results

Experiment 3.2 using Bi-RNN or Bidirectional LSTM

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 110.2888
R-squared (R²): -0.0449
Mean Squared Error (MSE): 12163.6298
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 4.9293
R-squared (R²): 0.8357
Mean Squared Error (MSE): 24.2980

```

Fig. 44. Experiment 3.2 - AMZN & IBM - Testing Results

```

Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 114.4370
R-squared (R²): 0.6794
Mean Squared Error (MSE): 13095.8272
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 25.1852
R-squared (R²): 0.5814
Mean Squared Error (MSE): 634.2961

```

Fig. 45. Experiment 3.2 - AMZN & IBM - Training Results

Experiment 4 : Changing Number of Units

Experiment 4.1 using 10 units on an LSTM model

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 95.7582
  R-squared (R²): 0.2123
  Mean Squared Error (MSE): 9169.6275
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 3.4583
  R-squared (R²): 0.9191
  Mean Squared Error (MSE): 11.9599
```

Fig. 46. Experiment 4.1 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 113.3245
  R-squared (R²): 0.6856
  Mean Squared Error (MSE): 12842.4341
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 23.8668
  R-squared (R²): 0.6241
  Mean Squared Error (MSE): 569.6233
```

Fig. 47. Experiment 4.1 - AMZN & IBM - Training Results

Experiment 4.2 using 20 units on an LSTM model

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 78.2340
  R-squared (R²): 0.4742
  Mean Squared Error (MSE): 6120.5557
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 3.3022
  R-squared (R²): 0.9263
  Mean Squared Error (MSE): 10.9042
```

Fig. 48. Experiment 4.2 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 111.8343
  R-squared (R²): 0.6938
  Mean Squared Error (MSE): 12506.9037
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 24.3433
  R-squared (R²): 0.6089
  Mean Squared Error (MSE): 592.5968
```

Fig. 49. Experiment 4.2 - AMZN & IBM - Training Results

Experiment 4.3 using 50 units on an LSTM model

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 64.0059
  R-squared (R²): 0.6481
  Mean Squared Error (MSE): 4096.7588
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 4.5496
  R-squared (R²): 0.8600
  Mean Squared Error (MSE): 20.6990
```

Fig. 50. Experiment 4.3 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 116.0448
  R-squared (R²): 0.6703
  Mean Squared Error (MSE): 13466.3993
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 25.3823
  R-squared (R²): 0.5748
  Mean Squared Error (MSE): 644.2602
```

Fig. 51. Experiment 4.3 - AMZN & IBM - Training Results

Experiment 5 : Changing Learning Rate

Experiment 5 Changing Learning Rate from 0.001 to 0.0001

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 300.0399
  R-squared (R²): -6.7335
  Mean Squared Error (MSE): 90023.9480
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 8.0022
  R-squared (R²): 0.5670
  Mean Squared Error (MSE): 64.0346
```

Fig. 52. Experiment 5 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
  Root Mean Squared Error (RMSE): 93.3731
  R-squared (R²): 0.7866
  Mean Squared Error (MSE): 8718.5376
Performance metrics for IBM:
  Root Mean Squared Error (RMSE): 25.1853
  R-squared (R²): 0.5814
  Mean Squared Error (MSE): 634.3001
```

Fig. 52. Experiment 5 - AMZN & IBM - Training Results

Experiment 6 : Different Sequence Lengths (Time Steps)

Experiment 6.1 Using 30 Time Steps Length

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 87.5677
R-squared (R²): 0.3413
Mean Squared Error (MSE): 7668.0976
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 3.5669
R-squared (R²): 0.9140
Mean Squared Error (MSE): 12.7227
```

Fig. 53. Experiment 6.1 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 125.7248
R-squared (R²): 0.6130
Mean Squared Error (MSE): 15806.7153
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.3513
R-squared (R²): 0.6087
Mean Squared Error (MSE): 592.9854
```

Fig. 54. Experiment 6.1 - AMZN & IBM - Training Results

Experiment 6.2 Using 90 Time Steps Length

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 76.8840
R-squared (R²): 0.4922
Mean Squared Error (MSE): 5911.1534
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 4.2005
R-squared (R²): 0.8807
Mean Squared Error (MSE): 17.6440
```

Fig. 55. Experiment 6.2 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 130.5479
R-squared (R²): 0.5828
Mean Squared Error (MSE): 17042.7594
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 25.0013
R-squared (R²): 0.5875
Mean Squared Error (MSE): 625.0665
```

Fig. 56. Experiment 6.2 - AMZN & IBM - Training Results

Experiment 7 : Dropout Regularization

Experiment 7 using 20% dropout rate

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 251.7860
R-squared (R²): -4.4461
Mean Squared Error (MSE): 63396.1690
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 16.4178
R-squared (R²): -0.8225
Mean Squared Error (MSE): 269.5442
```

Fig. 57. Experiment 7 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 69.4989
R-squared (R²): 0.8818
Mean Squared Error (MSE): 4830.0993
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.6720
R-squared (R²): 0.5983
Mean Squared Error (MSE): 608.7077
```

Fig. 58. Experiment 7 - AMZN & IBM - Training Results

Experiment 8 : Momentum-Based Optimization

Experiment 8 using Adam optimizer

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 108.9842
R-squared (R²): -0.0203
Mean Squared Error (MSE): 11877.5658
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 3.9653
R-squared (R²): 0.8937
Mean Squared Error (MSE): 15.7233
```

Fig. 59. Experiment 8 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 116.7352
R-squared (R²): 0.6664
Mean Squared Error (MSE): 13627.1041
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 23.5063
R-squared (R²): 0.6353
Mean Squared Error (MSE): 552.5466
```

Fig. 60. Experiment 8 - AMZN & IBM - Training Results

Experiment 9 : Data Augmentation

Experiment 9 using Rolling Average before Scaling

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 32.9150
R-squared (R²): 0.9069
Mean Squared Error (MSE): 1083.4000
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 4.0706
R-squared (R²): 0.8880
Mean Squared Error (MSE): 16.5696
```

Fig. 61. Experiment 9 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 142.6055
R-squared (R²): 0.5022
Mean Squared Error (MSE): 20336.3370
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.7311
R-squared (R²): 0.5964
Mean Squared Error (MSE): 611.6292
```

Fig. 62. Experiment 9 - AMZN & IBM - Training Results

Experiment 10 : Feature Engineering

Experiment 10.1 Feature Engineering (Open Column)

```
Performance metrics for AMZN (Open):
Root Mean Squared Error (RMSE): 161.9100
R-squared (R²): -1.2914
Mean Squared Error (MSE): 26214.8619
Performance metrics for IBM (Open):
Root Mean Squared Error (RMSE): 4.5267
R-squared (R²): 0.8589
Mean Squared Error (MSE): 20.4908
```

Fig. 63. Experiment 10.1 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN (Open):
Root Mean Squared Error (RMSE): 115.6929
R-squared (R²): 0.6674
Mean Squared Error (MSE): 13384.8458
Performance metrics for IBM (Open):
Root Mean Squared Error (RMSE): 24.9482
R-squared (R²): 0.5876
Mean Squared Error (MSE): 622.4133
```

Fig. 64. Experiment 10.1 - AMZN & IBM - Training Results

Experiment 10.2 Feature Engineering (High Column)

```
Performance metrics for AMZN (High):
Root Mean Squared Error (RMSE): 171.6560
R-squared (R²): -1.5313
Mean Squared Error (MSE): 29465.7838
Performance metrics for IBM (High):
Root Mean Squared Error (RMSE): 5.8374
R-squared (R²): 0.7696
Mean Squared Error (MSE): 34.0753
```

Fig. 65. Experiment 10.2 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN (High):
Root Mean Squared Error (RMSE): 114.4212
R-squared (R²): 0.6795
Mean Squared Error (MSE): 13092.2002
Performance metrics for IBM (High):
Root Mean Squared Error (RMSE): 24.3719
R-squared (R²): 0.6080
Mean Squared Error (MSE): 593.9906
```

Fig. 66. Experiment 10.2 - AMZN & IBM - Training Results

Experiment 10.3 Feature Engineering (Low Column)

```
Performance metrics for AMZN (Low):
Root Mean Squared Error (RMSE): 152.5248
R-squared (R²): -1.0974
Mean Squared Error (MSE): 23263.8120
Performance metrics for IBM (Low):
Root Mean Squared Error (RMSE): 4.0590
R-squared (R²): 0.8848
Mean Squared Error (MSE): 16.4758
```

Fig. 67. Experiment 10.3 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN (Low):
Root Mean Squared Error (RMSE): 114.6913
R-squared (R²): 0.6670
Mean Squared Error (MSE): 13154.0860
Performance metrics for IBM (Low):
Root Mean Squared Error (RMSE): 24.7980
R-squared (R²): 0.5908
Mean Squared Error (MSE): 614.9408
```

Fig. 68. Experiment 10.3 - AMZN & IBM - Training Results

Experiment 10.4 Feature Engineering (Volume Column)

```
Performance metrics for AMZN (Volume):  
Root Mean Squared Error (RMSE): 2807468.3554  
R-squared ( $R^2$ ): -1.7033  
Mean Squared Error (MSE): 7881878566502.6816  
Performance metrics for IBM (Volume):  
Root Mean Squared Error (RMSE): 2788499.8001  
R-squared ( $R^2$ ): -0.1775  
Mean Squared Error (MSE): 7775731135186.0264
```

Fig. 69. Experiment 10.4 - AMZN & IBM - Testing Results

```
Performance metrics for AMZN (Volume):  
Root Mean Squared Error (RMSE): 4954445.2723  
R-squared ( $R^2$ ): 0.1156  
Mean Squared Error (MSE): 24546527955978.6680  
Performance metrics for IBM (Volume):  
Root Mean Squared Error (RMSE): 2946179.9814  
R-squared ( $R^2$ ): 0.1563  
Mean Squared Error (MSE): 8679976482608.6602
```

Fig. 70. Experiment 10.4 - AMZN & IBM - Training Results

Final Model (F)

Final Model - Stock Price Prediction Graph - Testing

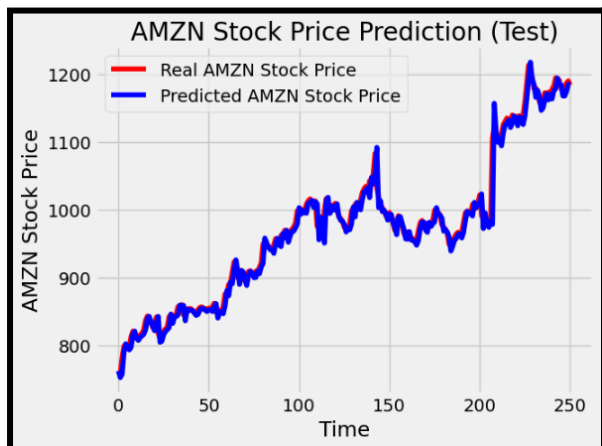


Fig. 71. AMZN Stock Price Prediction - Final Model - Testing Results - Stock Price Prediction Graph

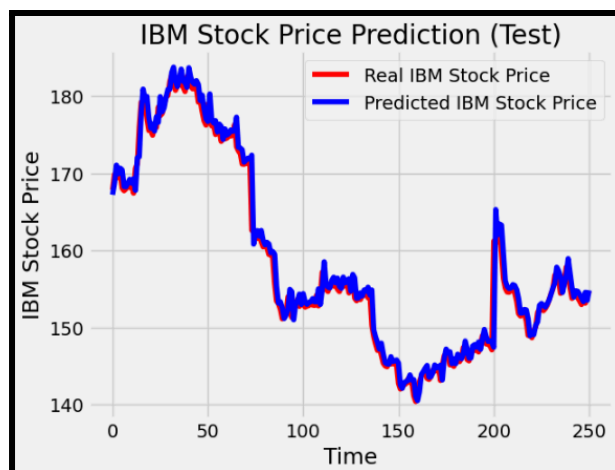


Fig. 72. IBM Stock Price Prediction - Final Model - Testing Results - Stock Price Prediction Graph

Final Model - Stock Price Prediction Graph - Training

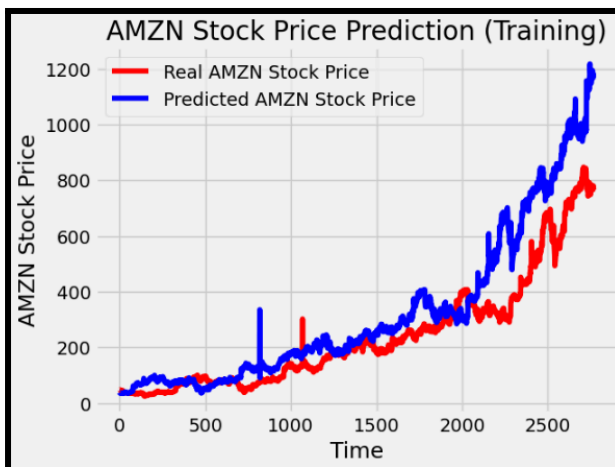


Fig. 73. AMZN Stock Price Prediction - Final Model - Training Results - Stock Price Prediction Graph

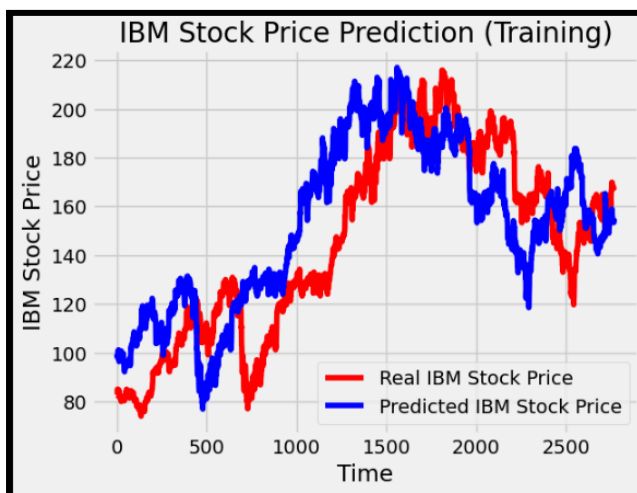


Fig. 74. IBM Stock Price Prediction - Final Model - Training Results - Stock Price Prediction Graph

Final Model - Performance Metrics Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 13.7966
R-squared (R²): 0.9836
Mean Squared Error (MSE): 190.3452
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 1.6757
R-squared (R²): 0.9810
Mean Squared Error (MSE): 2.8079
```

Fig. 75. Final Model - AMZN & IBM - Testing Results

```
Performance metrics for AMZN:
Root Mean Squared Error (RMSE): 130.7515
R-squared (R²): 0.5815
Mean Squared Error (MSE): 17095.9628
Performance metrics for IBM:
Root Mean Squared Error (RMSE): 24.9080
R-squared (R²): 0.5906
Mean Squared Error (MSE): 620.4069
```

Fig. 76. Final Model - AMZN & IBM - Training Results

B. Compiled Model Experiment Results

TABLE 6

Results of Experiments for AMZN stock price history dataset

Experiment	Training	Testing
Baseline	RMSE: 106.52	RMSE: 116.31
	R²: 0.72	R²: -0.16
	MSE: 11346.84	MSE: 13528.79
Experiment 1	RMSE: 115.44	RMSE: 174.56
	R²: 0.67	R²: -1.61
	MSE: 13326.46	MSE: 30473.95
Experiment 2	RMSE: 113.38	RMSE: 130.55
	R²: 0.69	R²: -0.46
	MSE: 12855.80	MSE: 17043.64
Experiment 3.1	RMSE: 111.61	RMSE: 88.38
	R²: 0.70	R²: 0.33
	MSE: 12456.65	MSE: 7812.49
Experiment 3.2	RMSE: 114.44	RMSE: 110.29
	R²: 0.68	R²: -0.04
	MSE: 13095.83	MSE: 12163.62
Experiment 4.1	RMSE: 11.95	RMSE: 95.75
	R²: 0.68	R²: 0.21
	MSE: 12842.43	MSE: 9169.63
Experiment 4.2	RMSE: 111.83	RMSE: 78.23
	R²: 0.69	R²: 0.47
	MSE: 12506.90	MSE: 6120.55
Experiment 4.3	RMSE: 116.04	RMSE: 64.01
	R²: 0.67	R²: 0.65
	MSE: 13466.40	MSE: 4096.76

Experiment 5	RMSE: 93.37	RMSE: 300.04
	R²: 0.78	R²: -6.73
	MSE: 8718.53	MSE: 90023.94
Experiment 6.1	RMSE: 125.72	RMSE: 87.56
	R²: 0.61	R²: 0.34
	MSE: 15806.71	MSE: 7668.10
Experiment 6.2	RMSE: 130.55	RMSE: 76.88
	R²: 0.58	R²: 0.49
	MSE: 17042.76	MSE: 5911.15
Experiment 7	RMSE: 69.50	RMSE: 251.79
	R²: 0.88	R²: -4.45
	MSE: 4830.10	MSE: 63396.17
Experiment 8	RMSE: 116.74	RMSE: 108.98
	R²: 0.67	R²: -0.02
	MSE: 13627.10	MSE: 11877.57
Experiment 9	RMSE: 142.61	RMSE: 32.92
	R²: 0.50	R²: 0.91
	MSE: 20336.33	MSE: 1083.40
Experiment 10 (Open)	RMSE: 115.69	RMSE: 161.91
	R²: 0.67	R²: -1.29
	MSE: 13384.85	MSE: 26214.86
Experiment 10 (High)	RMSE: 114.42	RMSE: 171.66
	R²: 0.68	R²: -1.53
	MSE: 13092.20	MSE: 29465.78
Experiment 10 (Low)	RMSE: 114.69	RMSE: 152.52
	R²: 0.67	R²: -1.10
	MSE: 13154.09	MSE: 23263.81
Experiment 10 (Volume)	RMSE: 4954445.27	RMSE: 2807468.36
	R²: 0.12	R²: -1.70
	MSE: 24546527955978.67	MSE: 7881878566502.68
Final Model	RMSE: 130.75	RMSE: 13.80
	R²: 0.58	R²: 0.98
	MSE: 7095.96	MSE: 190.35

Note: The Numbers are rounded up by 2 decimals

TABLE 7

Results of Experiments for IBM stock price history dataset

Experiment	Training	Testing
Baseline	RMSE: 24.45	RMSE: 4.23
	R²: 0.60	R²: 0.87
	MSE: 597.87	MSE: 17.91
Experiment 1	RMSE: 24.21	RMSE: 3.10
	R²: 0.61	R²: 0.93
	MSE: 586.01	MSE: 9.67
Experiment 2	RMSE: 23.42	RMSE: 5.30
	R²: 0.64	R²: 0.81
	MSE: 548.71	MSE: 28.07
Experiment 3.1	RMSE: 24.54	RMSE: 2.71

Experiment 3.2	R ² : 0.60	R ² : 0.95
	MSE: 602.19	MSE: 7.34
	RMSE: 25.19	RMSE: 4.93
	R ² : 0.58	R ² : 0.84
Experiment 4.1	MSE: 634.30	MSE: 24.30
	RMSE: 23.86	RMSE: 3.46
	R ² : 0.62	R ² : 0.92
	MSE: 569.62	MSE: 11.96
Experiment 4.2	RMSE: 24.34	RMSE: 3.30
	R ² : 0.60	R ² : 0.92
	MSE: 592.59	MSE: 10.90
	RMSE: 25.38	RMSE: 4.55
Experiment 4.3	R ² : 0.57	R ² : 0.86
	MSE: 644.26	MSE: 20.69
	RMSE: 25.18	RMSE: 8.00
	R ² : 0.58	R ² : 0.57
Experiment 5	MSE: 634.30	MSE: 64.03
	RMSE: 24.35	RMSE: 3.57
	R ² : 0.60	R ² : 0.91
	MSE: 592.98	MSE: 12.72
Experiment 6.1	RMSE: 25.00	RMSE: 4.20
	R ² : 0.58	R ² : 0.88
	MSE: 625.07	MSE: 17.64
	RMSE: 24.67	RMSE: 16.42
Experiment 6.2	R ² : 0.60	R ² : -0.82
	MSE: 608.71	MSE: 269.54
	RMSE: 23.51	RMSE: 3.97
	R ² : 0.64	R ² : 0.89
Experiment 7	MSE: 552.55	MSE: 15.72
	RMSE: 24.73	RMSE: 4.07
	R ² : 0.60	R ² : 0.89
	MSE: 611.63	MSE: 16.57
Experiment 8	RMSE: 24.95	RMSE: 4.53
	R ² : 0.59	R ² : 0.86
	MSE: 622.41	MSE: 20.49
	RMSE: 24.37	RMSE: 5.84
Experiment 9	R ² : 0.61	R ² : 0.77
	MSE: 593.99	MSE: 34.08
	RMSE: 24.80	RMSE: 4.06
	R ² : 0.59	R ² : 0.89
Experiment 10 (Open)	MSE: 614.94	MSE: 16.48
	RMSE: 2946179.98	RMSE: 2788499.80
	R ² : 0.16	R ² : -0.18
	MSE: 8679976482608.66	MSE: 7775731135186.03
Experiment 10 (High)	RMSE: 24.91	RMSE: 1.68
	R ² : 0.59	R²: 0.98
	MSE: 620.41	MSE: 2.81
Experiment 10 (Low)		
Experiment 10 (Volume)		
Final Model		

Note: The Numbers are rounded up by 2 decimals

The experiments on the AMZN and IBM stock price datasets tested several hyperparameters, architectures, and preprocessing methods to maximize RNN performance. Important findings indicated that GRUs (Experiment 3.1) performed better than LSTMs and bidirectional RNNs, with the best testing RMSE (88.38 for AMZN, 2.71 for IBM), which confirmed their effectiveness for stock price forecasting. Bigger units (50 for AMZN, 20 for IBM in Experiment 4) enhanced test performance, implying greater model capacity is a good thing, but tuning must be done cautiously to prevent overfitting. Augmentation by smoothing (Experiment 9) significantly helped generalization, lowering test RMSE to 32.92 for AMZN and 4.07 for IBM, whereas raw features such as 'Open' or 'High' (Experiment 10) did not do well without preprocessing. Adaptive optimizers such as Adam (implicit in the final model) were more resilient than methods based on momentum (Experiment 8), and longer sequence lengths (90 time steps in Experiment 6) captured more elaborate temporal context.

The best-performing techniques were combined in the final model: smoothed input data (rolling average with window=3), normalization using MinMaxScaler, and GRU layers (2 layers with 50 units for AMZN, 20 for IBM) trained on Adam (learning rate=0.01). This design attained outstanding performance, with test RMSE of 13.80 (R²=0.98) for AMZN and 1.68 (R²=0.98) for IBM. The success highlights the value of preprocessing (smoothing and scaling) and GRUs' capacity to strike a balance between complexity and efficiency. Hyperparameter tuning through grid search, ensemble techniques with Transformers, and targeted regularization could further improve performance in future work. Feature importance analysis, particularly for metrics such as trading volume, could also provide actionable insights. In general, the experiments show that rigorous optimization of architecture and treatment of data can provide very accurate predictions of stock prices.

V. CONCLUSION

This research aimed to solve the issue of predicting future stock prices for Amazon (AMZN) and IBM based on past data, with the intention of developing an efficient Recurrent Neural Network (RNN) model from scratch and testing different configurations to achieve maximum performance. By a sequence of systematic experiments, the study met its goals: deploying a baseline LSTM model, adjusting major hyperparameters, investigating various types of RNN (such as GRU and Bi-RNN), and using various preprocessing methods like normalization, dropout, data

augmentation, and feature engineering. Among the results, the GRU-based model performed significantly better than other structures in test accuracy, especially when augmented with rolling average smoothing and MinMax scaling. The last model, a well-tuned GRU network with 50 and 20 units for AMZN and IBM respectively, trained using the Adam optimizer, performed exceptionally well—with R^2 values of 0.98 for both stocks and RMSE values of 13.80 for AMZN and 1.68 for IBM. These results affirm GRU efficacy in time-series forecasting and underscore that efficient data treatment, tuning, and choice of architecture greatly improve model performance. In the end, this research underscores the fact that RNNs—once carefully optimized—are reliable instruments for stock price prediction, giving investors useful knowledge to inform financially sound decision-making.

VI. REFERENCE

- [1] Stryker, C. (2024, October 4). Recurrent Neural Network (RNN). How RNNs work. <https://www.ibm.com/think/topics/recurrent-neural-networks>
- [2] Idrees, H. (2024, November 15). CNN vs. RNN: Understanding Their Roles in Image and Sequential Data Processing. Medium. <https://medium.com/@hassaanidrees7/cnn-vs-rnn-understanding-their-roles-in-image-and-sequential-data-processing-65684cc05902>
- [3] What Is Data Imputation: Purpose, Techniques, & Methods. (2024, April 30). <https://airbyte.com/data-engineering-resources/data-imputation#:~:text=Data%20imputation%20is%20the%20process,the%20quality%20of%20the%20dataset.>
- [4] Frost F. (n.d.). Mean Squared Error (MSE). Statistics by Jim. <https://statisticsbyjim.com/regression/mean-squared-error-mse/>
- [5] Frost F. (n.d.). Root Mean Squared Error (RMSE). Statistics by Jim. <https://statisticsbyjim.com/regression/root-mean-square-error-rmse/>
- [6] Fernando, J. (2024, November 13). R-Squared: Definition, Calculation, and interpretation. Investopedia. <https://www.investopedia.com/terms/r/r-squared.asp>